

✓ Carregamento dos dados

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("olistbr/brazilian-ecommerce")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'brazilian-ecommerce' dataset.
Path to dataset files: /kaggle/input/brazilian-ecommerce

```
import os

# Lista os arquivos no diretório do dataset
file_list = os.listdir(path)

print("Arquivos no diretório do dataset:")
for file_name in file_list:
    print(file_name)
```

Arquivos no diretório do dataset:

- olist_customers_dataset.csv
- olist_sellers_dataset.csv
- olist_order_reviews_dataset.csv
- olist_order_items_dataset.csv
- olist_products_dataset.csv
- olist_geolocation_dataset.csv
- product_category_name_translation.csv
- olist_orders_dataset.csv
- olist_order_payments_dataset.csv

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("olistbr/brazilian-ecommerce")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'brazilian-ecommerce' dataset.
Path to dataset files: /kaggle/input/brazilian-ecommerce

```
import os

# Lista os arquivos no diretório do dataset
file_list = os.listdir(path)

print("Arquivos no diretório do dataset:")
for file_name in file_list:
    print(file_name)
```

Arquivos no diretório do dataset:

- olist_customers_dataset.csv
- olist_sellers_dataset.csv
- olist_order_reviews_dataset.csv
- olist_order_items_dataset.csv
- olist_products_dataset.csv
- olist_geolocation_dataset.csv
- product_category_name_translation.csv
- olist_orders_dataset.csv
- olist_order_payments_dataset.csv

```
# Renomear os arquivos para remover 'olist_' e '_dataset'
renamed_file_list = []
for file_name in file_list:
    new_name = file_name.replace('olist_', '').replace('_dataset', '')
    renamed_file_list.append(new_name)

print("Arquivos renomeados:")
for file_name in renamed_file_list:
    print(file_name)
```

```
Arquivos renomeados:
customers.csv
sellers.csv
order_reviews.csv
order_items.csv
products.csv
geolocation.csv
product_category_name_translation.csv
orders.csv
order_payments.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

▼ Query do arquivo Logistica.CSV

```
SELECT
  o.order_id,
  o.order_purchase_t,
  o.order_delivered_6,
  o.order_estimated,
  c.customer_state AS uf_cliente,
  p.product_category AS categoria,
  s.seller_id
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
JOIN order_items AS i ON o.order_id = i.order_id
JOIN products AS p ON i.product_id = p.product_id
JOIN sellers AS s ON i.seller_id = s.seller_id
WHERE o.order_status = 'delivered';
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
df = pd.read_csv('/logistica.csv', index_col=0)
```

▼ 1:Distribuição do Tempo de Entrega

```
df['order_purchase_t'] = pd.to_datetime(df['order_purchase_t'])
df['order_delivered_6'] = pd.to_datetime(df['order_delivered_6'])
df['order_estimated'] = pd.to_datetime(df['order_estimated'])

# Cálculo do tempo de entrega em dias
df['tempo_entrega_dias'] = (df['order_delivered_6'] - df['order_purchase_t']).dt.days
df = df.dropna(subset=['tempo_entrega_dias'])

# Cria uma tabela apenas com os dados válidos e as colunas necessárias
tabela_tempo_entrega = df.dropna(subset=['tempo_entrega_dias'])[['order_id', 'tempo_entrega_dias']]

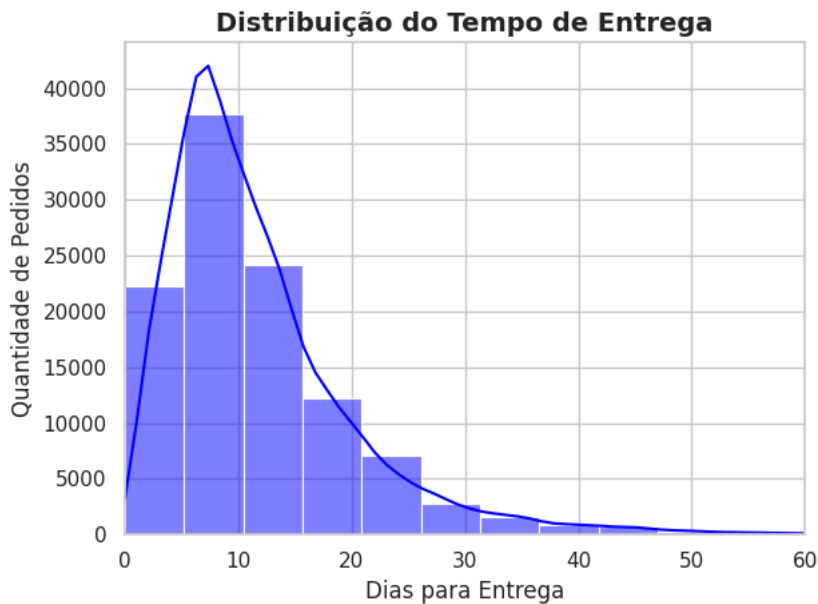
# Exibe as primeiras linhas da tabela
display(tabela_tempo_entrega.head())
```

	order_id	tempo_entrega_dias
0	e481f51cbdc54678b7cc49136f2d6af7	8
1	53cdb2fc8bc7dce0b6741e2150273451	13
2	47770eb9100c2d0c44946d9cf07ec65d	9
3	949d5b44dbf5de918fe9c16f97b45f8a	13
4	ad21c59c0840e6cb83a9ceb5573f8159	2

```
sns.histplot(data=tabela_tempo_entrega, x='tempo_entrega_dias', bins=40, kde=True, color='blue')

plt.title('Distribuição do Tempo de Entrega', fontsize=14, fontweight='bold')
plt.xlabel('Dias para Entrega', fontsize=12)
plt.ylabel('Quantidade de Pedidos', fontsize=12)
plt.xlim(0, 60)

plt.tight_layout()
plt.show()
```



2:Variação do Tempo por Região/UF

```
df['tempo_entrega_dias'] = (df['order_delivered_6'] - df['order_purchase_t']).dt.days
df = df.dropna(subset=['tempo_entrega_dias'])

# Ordenar os estados pelo tempo mediano de entrega
ordem_uf = df.groupby('uf_cliente')['tempo_entrega_dias'].median().sort_values(ascending=False).index

# Exibir uma tabela resumo das medianas por UF
tabela_mediana_uf = tabela_uf_tempo.groupby('uf_cliente')['tempo_entrega_dias'].median().sort_values(ascending=False).reset_index()
display(tabela_mediana_uf.head())
```

	uf_cliente	tempo_entrega_dias
0	AM	26.0
1	RR	24.5
2	AP	24.0
3	AL	22.0
4	PA	21.0

```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))

sns.boxplot(data=tabela_uf_tempo, x='uf_cliente', y='tempo_entrega_dias', order=ordem_uf, palette='viridis')

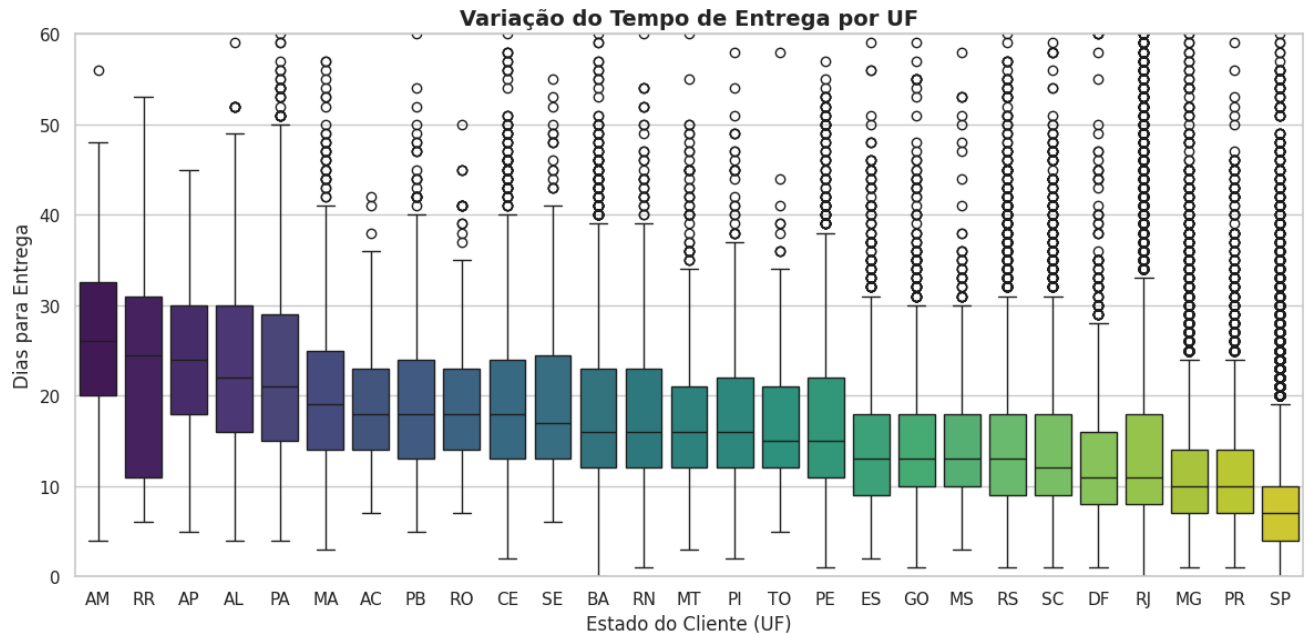
plt.title('Variação do Tempo de Entrega por UF', fontsize=14, fontweight='bold')
plt.xlabel('Estado do Cliente (UF)', fontsize=12)
plt.ylabel('Dias para Entrega', fontsize=12)
plt.ylim(0, 60)

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_5659/3604129648.py:4: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(data=tabela_uf_tempo, x='uf_cliente', y='tempo_entrega_dias', order=ordem_uf, palette='viridis')
```



✓ 3:% de Atraso por UF

```
# Identificar atraso (1 = Atrasado, 0 = No prazo)
df['atrasado'] = np.where(df['order_delivered_6'] > df['order_estimated'], 1, 0)

# Calcular a % de atraso por UF e ordenar do maior para o menor
atraso_por_uf = df.groupby('uf_cliente')['atrasado'].mean().sort_values(ascending=False) * 100

# Criar a tabela agrupada com a porcentagem de atraso por estado
tabela_atraso_uf = (df.groupby('uf_cliente')['atrasado'].mean() * 100).sort_values(ascending=False).reset_index()
tabela_atraso_uf.rename(columns={'atrasado': 'porcentagem_atraso'}, inplace=True)

# Exibe a tabela consolidada
display(tabela_atraso_uf.head())
```

	uf_cliente	porcentagem_atraso
0	AL	24.121780
1	MA	20.375000
2	SE	16.266667
3	PI	15.487572
4	CE	15.287518

```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))

sns.barplot(data=tabela_atraso_uf, x='uf_cliente', y='porcentagem_atraso', palette='Reds_r')

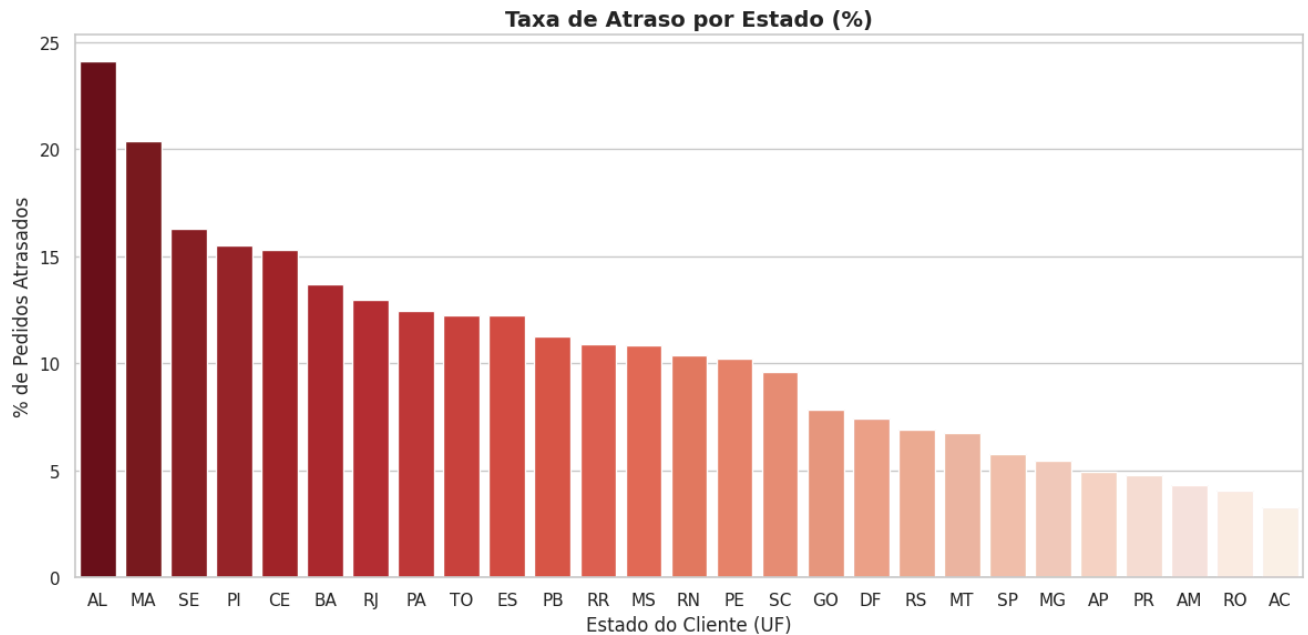
plt.title('Taxa de Atraso por Estado (%)', fontsize=14, fontweight='bold')
plt.xlabel('Estado do Cliente (UF)', fontsize=12)
plt.ylabel('% de Pedidos Atrasados', fontsize=12)

plt.tight_layout()
plt.show()
```

/tmp/ipykernel_5659/1485602728.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.barplot(data=tabela_atraso_uf, x='uf_cliente', y='porcentagem_atraso', palette='Reds_r')
```



4:Atraso ao Longo do Tempo

```
df['ano_mes'] = df['order_purchase_t'].dt.to_period('M')

# Criar a tabela agrupada com a evolução temporal
tabela_atraso_tempo = (df.groupby('ano_mes')['atrasado'].mean() * 100).reset_index()
tabela_atraso_tempo.rename(columns={'atrasado': 'porcentagem_atraso'}, inplace=True)

# Converter a coluna de data para texto para facilitar a visualização no gráfico
tabela_atraso_tempo['ano_mes'] = tabela_atraso_tempo['ano_mes'].astype(str)

# Exibe a tabela consolidada
display(tabela_atraso_tempo.head())
```

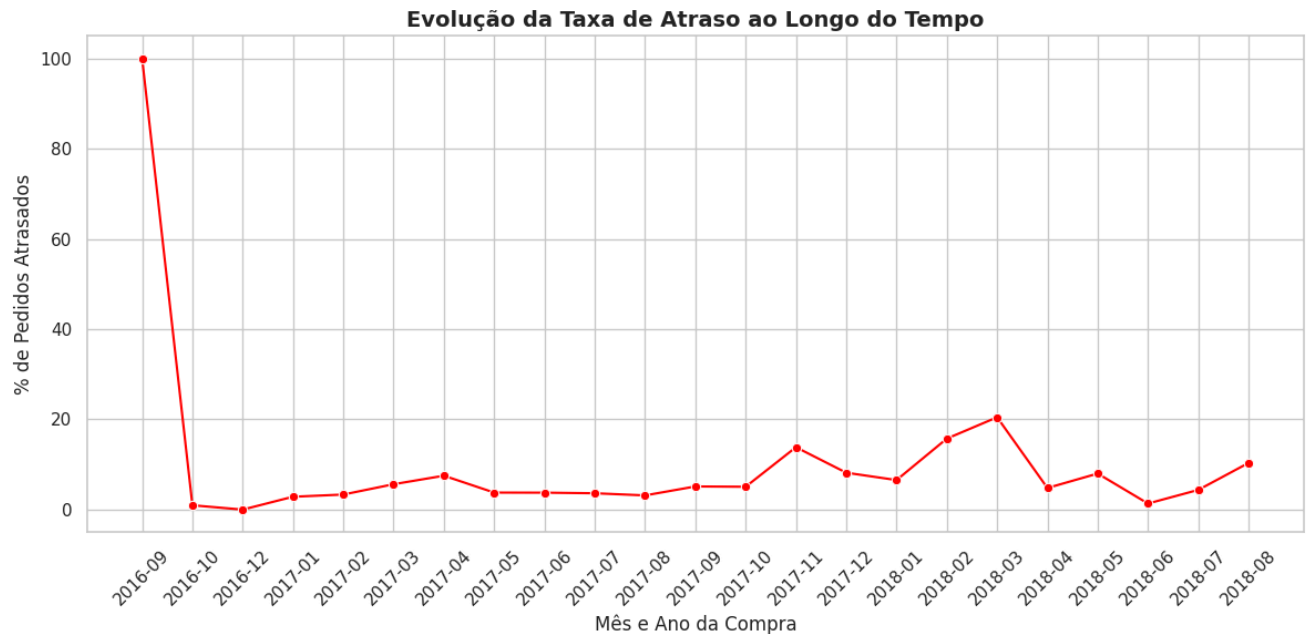
	ano_mes	porcentagem_atraso
0	2016-09	100.000000
1	2016-10	0.958466
2	2016-12	0.000000
3	2017-01	2.847755
4	2017-02	3.336921

```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))

sns.lineplot(data=tabela_atraso_tempo, x='ano_mes', y='porcentagem_atraso', color='red', marker='o')

plt.title('Evolução da Taxa de Atraso ao Longo do Tempo', fontsize=14, fontweight='bold')
plt.xlabel('Mês e Ano da Compra', fontsize=12)
plt.ylabel('% de Pedidos Atrasados', fontsize=12)
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



5: Atraso por Produto (Categoria)

```
# Calcular a % de atraso por categoria de produto e ordenar do maior para o menor
tabela_atraso_categoria = df.groupby('categoria')['atrasado'].mean().sort_values(ascending=False) * 100
tabela_atraso_categoria = tabela_atraso_categoria.reset_index()
tabela_atraso_categoria.rename(columns={'atrasado': 'porcentagem_atraso'}, inplace=True)

# Exibe a tabela consolidada (top 10 categorias com maior atraso)
display(tabela_atraso_categoria.head(10))
```

	categoria	porcentagem_atraso
0	casa_conforto_2	16.666667
1	moveis_colchao_e_estofado	13.513514
2	audio	12.707182
3	fashion_underwear_e_moda_praia	12.598425
4	artigos_de_natal	12.000000
5	livros_tecnicos	11.026616
6	casa_conforto	10.256410
7	construcao_ferramentas_iluminacao	9.966777
8	alimentos	9.819639
9	eletronicos	9.747160

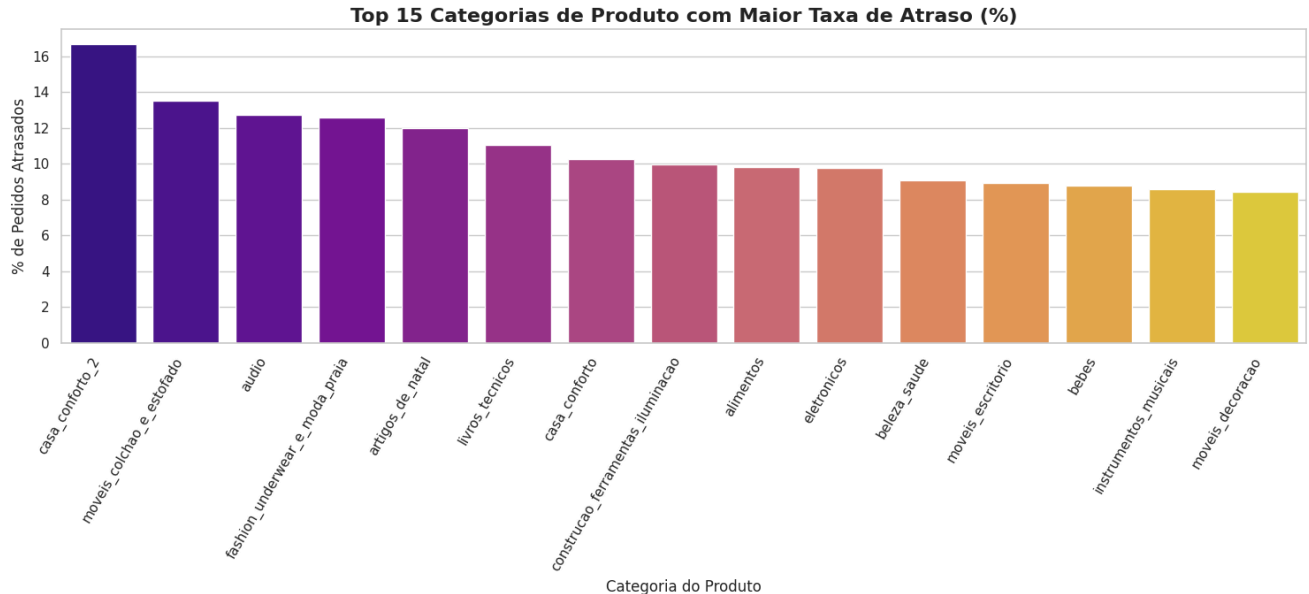
```
plt.figure(figsize=(15, 7))
sns.barplot(data=tabela_atraso_categoria.head(15), x='categoria', y='porcentagem_atraso', palette='plasma')

plt.title('Top 15 Categorias de Produto com Maior Taxa de Atraso (%)', fontsize=16, fontweight='bold')
plt.xlabel('Categoria do Produto', fontsize=12)
plt.ylabel('% de Pedidos Atrasados', fontsize=12)
plt.xticks(rotation=60, ha='right')
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_5659/2029238458.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.barplot(data=tabela_atraso_categoria.head(15), x='categoria', y='porcentagem_atraso', palette='plasma')
```



6: Atraso por Vendedor

```
# Calcular o número de pedidos por vendedor
contagem_pedidos_vendedor = df.groupby('seller_id')['order_id'].nunique()

# Filtrar vendedores com mais de 50 pedidos
vendedores_ativos = contagem_pedidos_vendedor[contagem_pedidos_vendedor > 50].index
df_vendedores_ativos = df[df['seller_id'].isin(vendedores_ativos)]

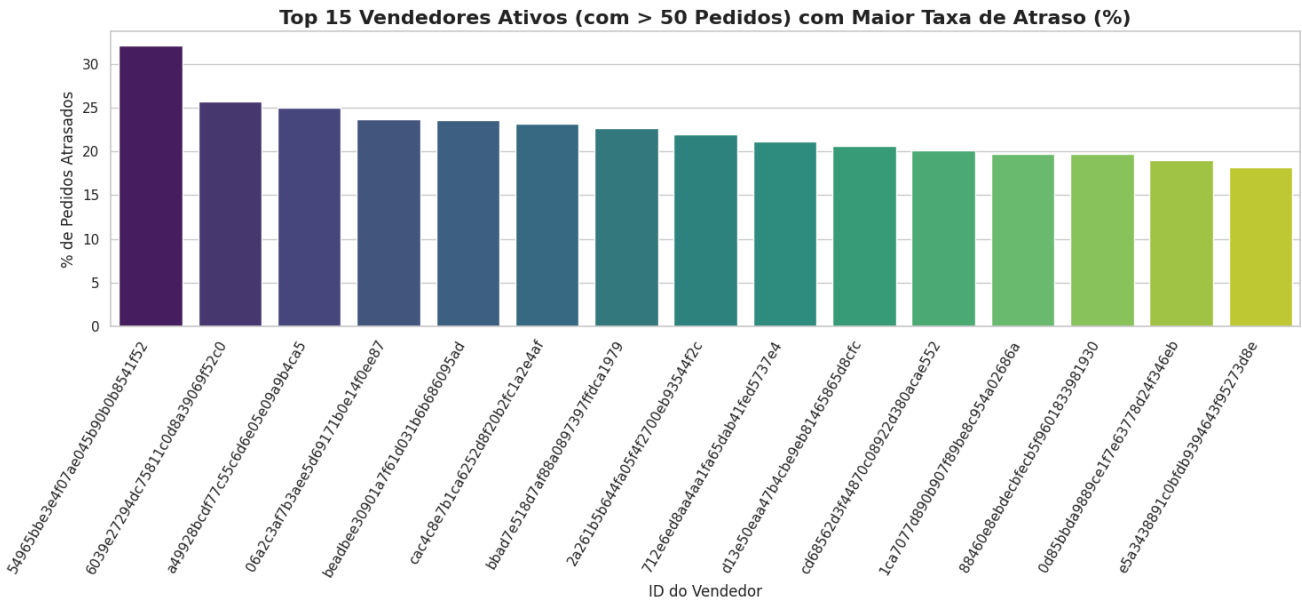
# Calcular a % de atraso por vendedor (apenas para vendedores ativos) e ordenar do maior para o menor
tabela_atraso_vendedor_filtrado = df_vendedores_ativos.groupby('seller_id')['atrasado'].mean().sort_values(ascending=False) * 100
tabela_atraso_vendedor_filtrado = tabela_atraso_vendedor_filtrado.reset_index()
tabela_atraso_vendedor_filtrado.rename(columns={'atrasado': 'porcentagem_atraso'}, inplace=True)

# Exibe a tabela consolidada (top 10 vendedores com maior atraso entre os ativos)
display(tabela_atraso_vendedor_filtrado.head(10))
```

	seller_id	porcentagem_atraso
0	54965bbe3e4f07ae045b90b0b8541f52	32.098765
1	6039e27294dc75811c0d8a39069f52c0	25.675676
2	a49928bcd77c55c6d6e05e09a9b4ca5	25.000000
3	06a2c3af7b3aee5d69171b0e14f0ee87	23.631841
4	beadbee30901a7f61d031b6b686095ad	23.529412
5	cac4c8e7b1ca6252d8f20b2fc1a2e4af	23.170732
6	bbad7e518d7af88a0897397ffdca1979	22.619048
7	2a261b5b644fa05f4f2700eb93544f2c	21.951220
8	712e6ed8aa4aa1fa65dab41fed5737e4	21.176471
9	d13e50eaa47b4cbe9eb81465865d8cfc	20.588235

```
plt.figure(figsize=(15, 7))
sns.barplot(data=tabela_atraso_vendedor_filtrado.head(15), x='seller_id', y='porcentagem_atraso', hue='seller_id', legend=False)

plt.title('Top 15 Vendedores Ativos (com > 50 Pedidos) com Maior Taxa de Atraso (%)', fontsize=16, fontweight='bold')
plt.xlabel('ID do Vendedor', fontsize=12)
plt.ylabel('% de Pedidos Atrasados', fontsize=12)
plt.xticks(rotation=60, ha='right')
plt.tight_layout()
plt.show()
```



```
sellers_df = pd.read_csv(os.path.join(path, 'olist_sellers_dataset.csv'))

# Merge com a tabela de atraso dos vendedores filtrados para obter a região do vendedor
tabela_final_vendedores = pd.merge(
    tabela_atraso_vendedor_filtrado.head(15),
    sellers_df[['seller_id', 'seller_state']],
    on='seller_id',
    how='left'
)
```



```
# Renomear colunas para melhor clareza
tabela_final_vendedores.rename(columns={'seller_state': 'regiao_vendedor'}, inplace=True)

# Exibir a tabela final
display(tabela_final_vendedores)
```

	seller_id	porcentagem_atraso	regiao_vendedor
0	54965bbe3e4f07ae045b90b0b8541f52	32.098765	PR
1	6039e27294dc75811c0d8a39069f52c0	25.675676	SP
2	a49928bcdf77c55c6d6e05e09a9b4ca5	25.000000	SP
3	06a2c3af7b3aee5d69171b0e14f0ee87	23.631841	MA
4	beadbee30901a7f61d031b6b686095ad	23.529412	SP
5	cac4c8e7b1ca6252d8f20b2fc1a2e4af	23.170732	SP
6	bbad7e518d7af88a0897397ffdca1979	22.619048	SP
7	2a261b5b644fa05f4f2700eb93544f2c	21.951220	SP
8	712e6ed8aa4aa1fa65dab41fed5737e4	21.176471	SC
9	d13e50eaa47b4cbe9eb81465865d8cfc	20.588235	SP
10	cd68562d3f44870c08922d380acae552	20.155039	SP
11	1ca7077d890b907f89be8c954a02686a	19.685039	SP
12	88460e8ebdecfbecb5f9601833981930	19.666667	PR
13	0d85bbda9889ce1f7e63778d24f346eb	18.965517	MG
14	e5a3438891c0bfbdb9394643f95273d8e	18.218623	SP

Analises de Logistica

```
tempo_medio = df['tempo_entrega_dias'].mean()
tempo_mediano = df['tempo_entrega_dias'].median()
taxa_atraso = df['atrasado'].mean() * 100
```

```
print("--- INDICADORES DE LOGÍSTICA ---")
print(f"Tempo Médio de Entrega: {tempo_medio:.1f} dias")
print(f"Tempo Mediano (Mediana): {tempo_mediano:.1f} dias")
print(f"Taxa Geral de Atraso: {taxa_atraso:.2f}%\n")
```

```
--- INDICADORES DE LOGÍSTICA ---
Tempo Médio de Entrega: 12.0 dias
Tempo Mediano (Mediana): 10.0 dias
Taxa Geral de Atraso: 7.91%
```

```
# 1. Carregar o dataset de pedidos completo para obter colunas de data adicionais
# 'path' é uma variável já definida no notebook que aponta para o diretório dos datasets.
orders_full_df = pd.read_csv(os.path.join(path, 'olist_orders_dataset.csv'))

# 2. Converter colunas de data para datetime no orders_full_df
orders_full_df['order_purchase_timestamp'] = pd.to_datetime(orders_full_df['order_purchase_timestamp'])
orders_full_df['order_approved_at'] = pd.to_datetime(orders_full_df['order_approved_at'])
orders_full_df['order_delivered_carrier_date'] = pd.to_datetime(orders_full_df['order_delivered_carrier_date'])
orders_full_df['order_delivered_customer_date'] = pd.to_datetime(orders_full_df['order_delivered_customer_date'])
orders_full_df['order_estimated_delivery_date'] = pd.to_datetime(orders_full_df['order_estimated_delivery_date'])

# 3. Fazer um merge do df existente com orders_full_df para adicionar as novas colunas de data
# Usar um left merge para manter todos os pedidos que já estão em 'df'
df_extended = pd.merge(
    df,
    orders_full_df[['order_id', 'order_approved_at', 'order_delivered_carrier_date']],
    on='order_id',
    how='left'
)

# 4. Calcular os novos tempos (em dias)
```

```
# Tempo de aprovação: da compra a aprovação do pagamento
df_extended['tempo_aprovacao_dias'] = (df_extended['order_approved_at'] - df_extended['order_purchase_t']).dt.days

# Tempo até a postagem: da aprovação à entrega à transportadora (carrier)
df_extended['tempo_postagem_dias'] = (df_extended['order_delivered_carrier_date'] - df_extended['order_approved_at']).dt.days

# Tempo de transporte: da postagem à entrega ao cliente
df_extended['tempo_transporte_dias'] = (df_extended['order_delivered_6'] - df_extended['order_delivered_carrier_date']).dt.days

# Tempo de atraso (dias): a diferença positiva entre a entrega real e a estimada, se houver atraso
# 'order_estimated' já está em df e corresponde a 'order_estimated_delivery_date'
df_extended['tempo_atraso_dias_calc'] = (df_extended['order_delivered_6'] - df_extended['order_estimated']).dt.days
# Atraso só deve ser positivo; se entregue antes ou no prazo, o atraso é 0
df_extended['tempo_atraso_dias_calc'] = df_extended['tempo_atraso_dias_calc'].apply(lambda x: x if x > 0 else 0)

# 5. Calcular a taxa de pedidos dentro do prazo
# A coluna 'atrasado' já indica se o pedido foi atrasado (1) ou não (0) com base em 'order_delivered_6' vs 'order_estimated'
df_extended['dentro_prazo'] = np.where(df_extended['atrasado'] == 0, 1, 0)

taxa_dentro_prazo_geral = df_extended['dentro_prazo'].mean() * 100

print(f"Taxa Geral de Pedidos Entregues Dentro do Prazo: {taxa_dentro_prazo_geral:.2f}%\n")

# 6. Analisar por região (uf_cliente)
# Remover NaNs que podem ter surgido nas novas colunas de tempo ou na UF para a análise regional
df_extended_cleaned = df_extended.dropna(subset=[
    'tempo_aprovacao_dias',
    'tempo_postagem_dias',
    'tempo_transporte_dias',
    'tempo_atraso_dias_calc',
    'dentro_prazo',
    'uf_cliente'
])

analise_por_regiao = df_extended_cleaned.groupby('uf_cliente').agg(
    tempo_medio_aprovacao=('tempo_aprovacao_dias', 'mean'),
    tempo_medio_postagem=('tempo_postagem_dias', 'mean'),
    tempo_medio_transporte=('tempo_transporte_dias', 'mean'),
    tempo_medio_total_entrega=('tempo_entrega_dias', 'mean'), # Usando a coluna 'tempo_entrega_dias' já existente
    tempo_medio_atraso_se_atrasado=('tempo_atraso_dias_calc', lambda x: x[x > 0].mean()), # Média dos atrasos reais, apenas para
    taxa_dentro_prazo=('dentro_prazo', lambda x: x.mean() * 100)
).sort_values(by='taxa_dentro_prazo', ascending=False)

print("\n--- Análise por Região (UF do Cliente) ---")
display(analise_por_regiao.round(2))
```

Taxa Geral de Pedidos Entregues Dentro do Prazo: 92.09%

--- Análise por Região (UF do Cliente) ---

1 to 25 of 27 entriesFilter🔍

uf_cliente	tempo_medio_aprovacao	tempo_medio_postagem	tempo_medio_transporte	tempo_medio_total_entrega	tempo_medio_atraso_se_atrasar
AP	0.52	2.32	24.14	27.75	9
RR	0.22	2.72	24.15	27.83	
MA	0.22	1.91	23.07	25.96	
AC	0.41	2.31	16.91	20.33	1
CE	0.25	2.78	17.34	20.98	1
SE	0.27	2.75	15.22	18.87	1
RN	0.25	2.48	11.32	14.69	1
RJ	0.26	2.28	15.7	18.93	1
PI	0.32	2.46	19.84	23.3	1
PA	0.33	2.06	11.84	14.95	
GO	0.31	2.38	15.4	18.77	1
BA	0.28	2.35	14.5	17.79	1
ES	0.29	2.5	11.75	15.19	
RS	0.32	2.27	11.44	14.71	1
MT	0.35	2.13	14.34	17.51	1
PB	0.42	2.54	16.46	20.12	
MA	0.39	2.78	17.35	21.21	
AL	0.33	2.66	20.36	23.99	
SC	0.3	2.42	11.1	14.52	
MG	0.26	2.34	8.26	11.51	
SP	0.25	2.29	5.06	8.26	
PR	0.29	2.35	8.16	11.48	
MS	0.26	2.27	11.88	15.11	
DF	0.28	2.35	9.22	12.5	

Show 25 per page
Ceará (15.29%)
Piauí (15.49%) e

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

12

1. Desempenho Geral e Eficiência:

- Operação de e-commerce analisada apresenta um bom nível de serviço geral, com uma taxa de pedidos entregues dentro do prazo de 92,09%.
- A taxa geral de atraso na operação é de 7,91%.
- O tempo médio de entrega dos pedidos é de 12,0 dias.
- O tempo mediano de entrega é de 10,0 dias, indicando que metade de todos os pedidos chega aos clientes em 10 dias ou menos.

2. Gargalos Regionais (Foco no Norte e Nordeste):

- Os estados com os maiores tempos medianos para entrega estão localizados nas regiões Norte e Nordeste do Brasil:
 - Amazonas (26,0 dias),
 - Roraima (24,5 dias),
 - Amapá (24,0 dias),
 - Alagoas (22,0 dias) e
 - Pará (21,0 dias).
- Os estados que sofrem com as maiores porcentagens de atraso também estão no Nordeste:
 - Alagoas (24,12%),
 - Maranhão (20,38%),
 - Sergipe (16,27%),

3. Impacto do Tipo de Produto (Categorias):

- As características do produto influenciam diretamente a logística, visto que certas categorias sofrem mais atrasos.
- As categorias com as maiores taxas de atraso são:
 - "casa_conforto_2" (16,67%),
 - "moveis_colchao_e_estofado" (13,51%),
 - "audio" (12,71%),
 - "fashion_underwear_e_moda_praia" (12,60%) e
 - "artigos_de_natal" (12,00%).
- O destaque para móveis sugere dificuldades no transporte de produtos volumosos.

4. Influência dos Vendedores nos Atrasos:

- Analisando vendedores com volume considerável (mais de 50 pedidos), nota-se que parceiros específicos comprometem fortemente o prazo.
- O vendedor com o pior desempenho (estado do PR) tem uma taxa de atraso de 32,10%.
- A grande maioria dos 15 vendedores ativos com as maiores taxas de atraso (que variam entre 18,22% e 25,68%) concentra-se no estado de São Paulo (SP).

5. Sazonalidade e Comportamento ao Longo do Tempo:

- A análise da evolução temporal mostra flutuações nas taxas de atraso. Observam-se picos de atraso no início do ano de 2018 (fevereiro e março registraram taxas de atraso na faixa de 15% a 20%), que caíram nos meses subsequentes